

CS4432: Database Systems II

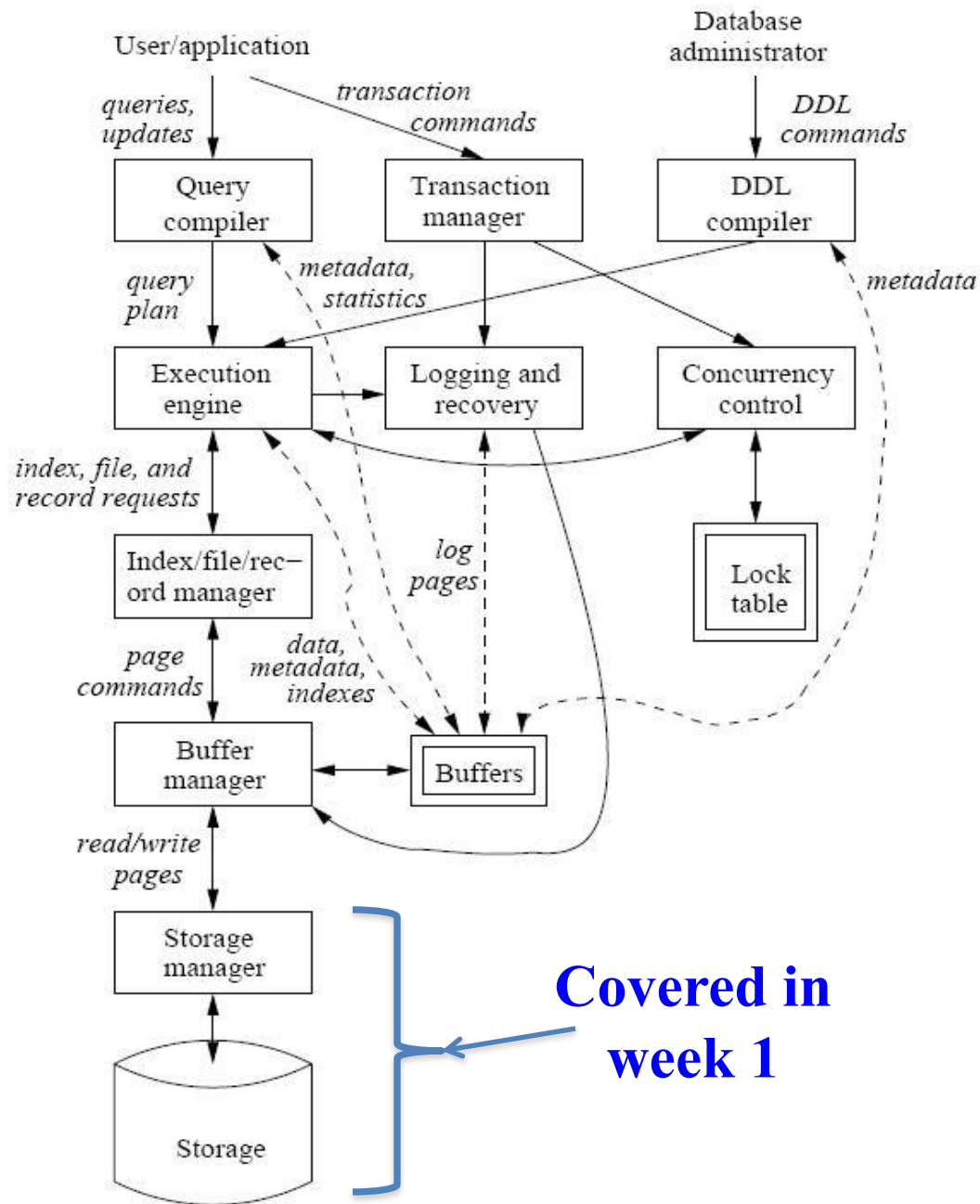
Buffer Manager (Part 1)

Updates

- **Tue (Mon schedule) March 31: Quiz 2**
You need to be in class
 - Comment on Quiz 1 (next slide)
- **HW1 is postponed to Friday**
 - make sure you submit in time.
- **Project 1** is out
 - Deadline: Apr 3 (wellness day Monday)

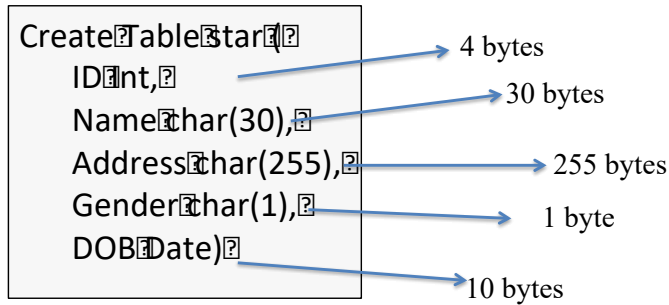
Quiz 1 Discussion

- **What did I hear?**
 - I have heard complaints about time and setup, anything else?
- **What about our grades?**
 - Make-up quizzes are coming – grades early next week. If factorization needed – will be provided.
- **What about next quizzes?**
 - Done via Canvas
 - 20 minutes instead of 15 minutes.



**Covered in
week 1**

Representing Tuples

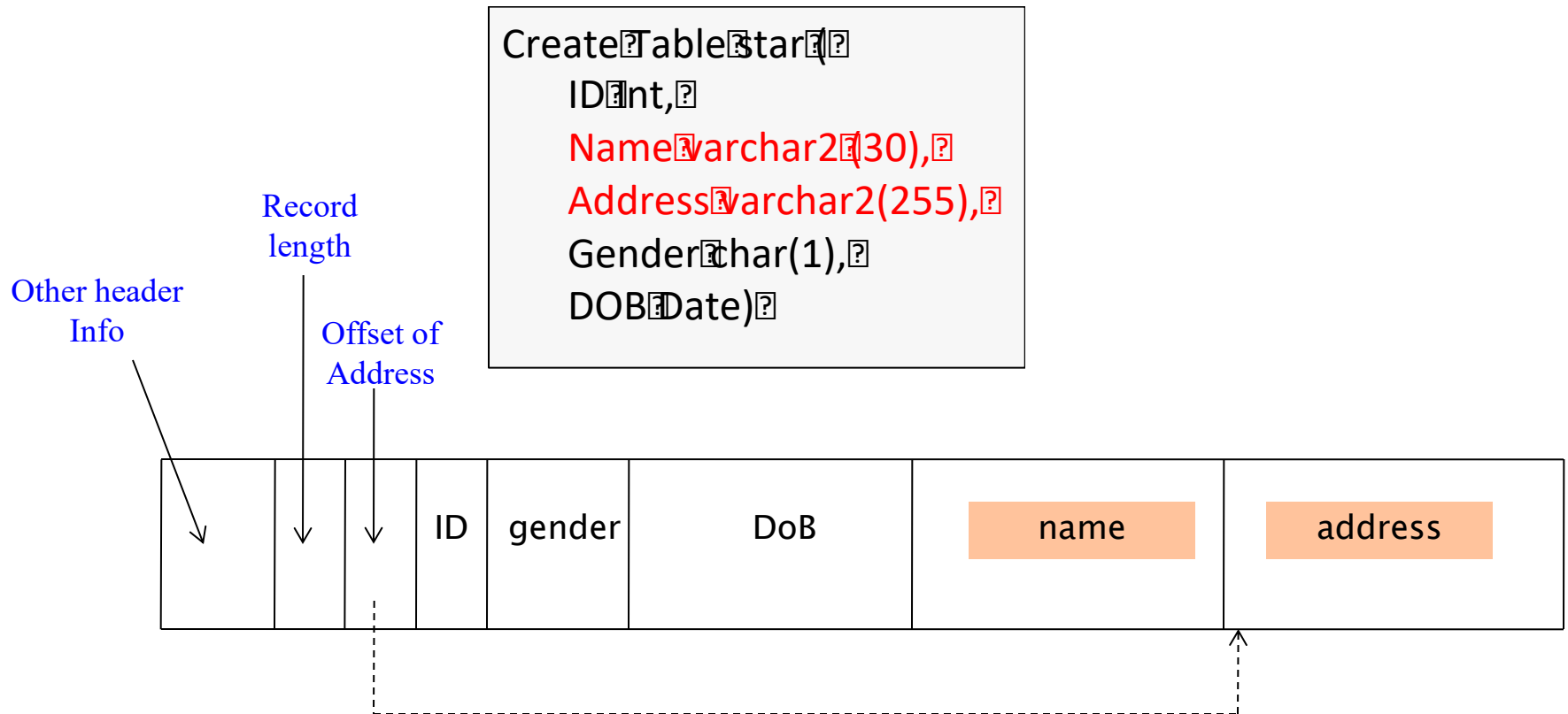


1- All fields are **aligned** to start at 4- or 8-byte boundaries (Hardware and OS requirements) & **concatenated**

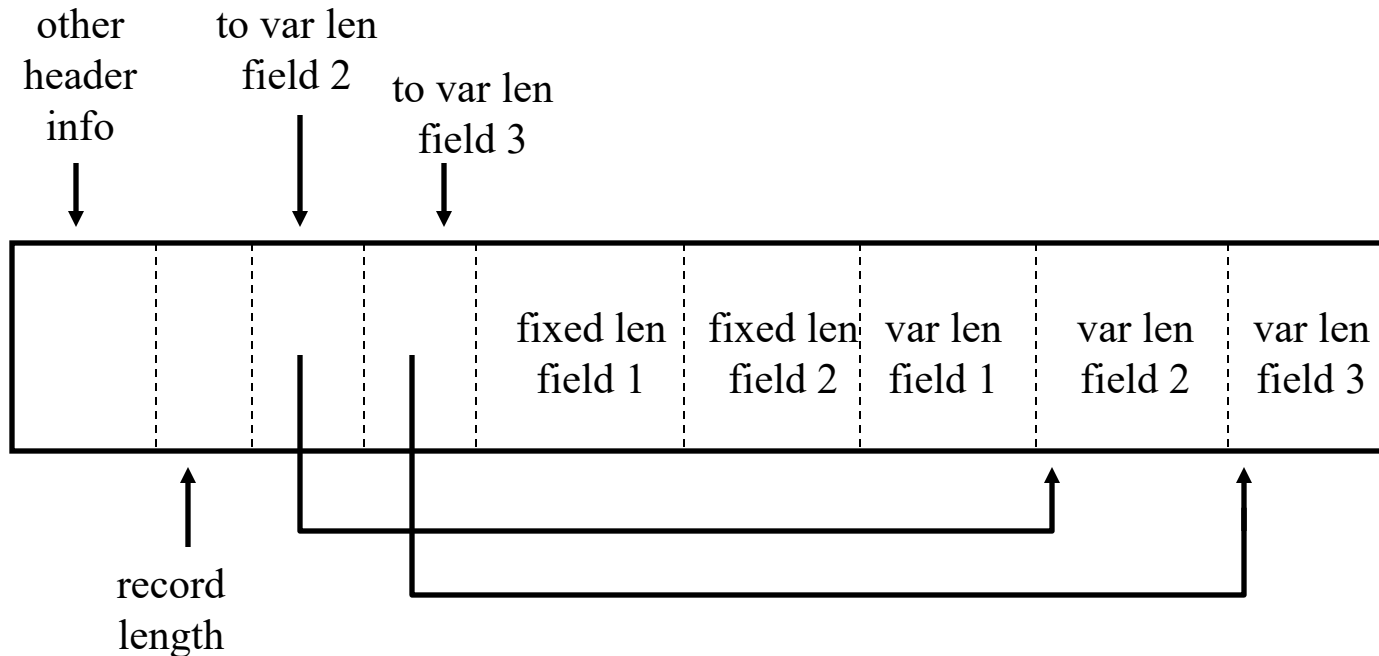
2- Each record has a header holding some info



Records with Variable-Length Fields



Extend to Multiple Fields

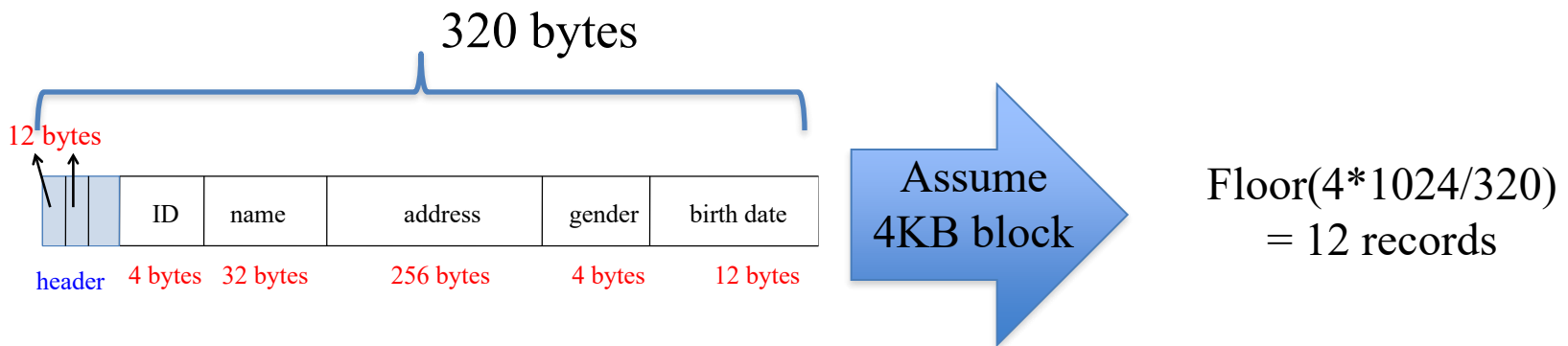


**Efficient access*

- Still reading the i^{th} field, does not require scanning over previous fields

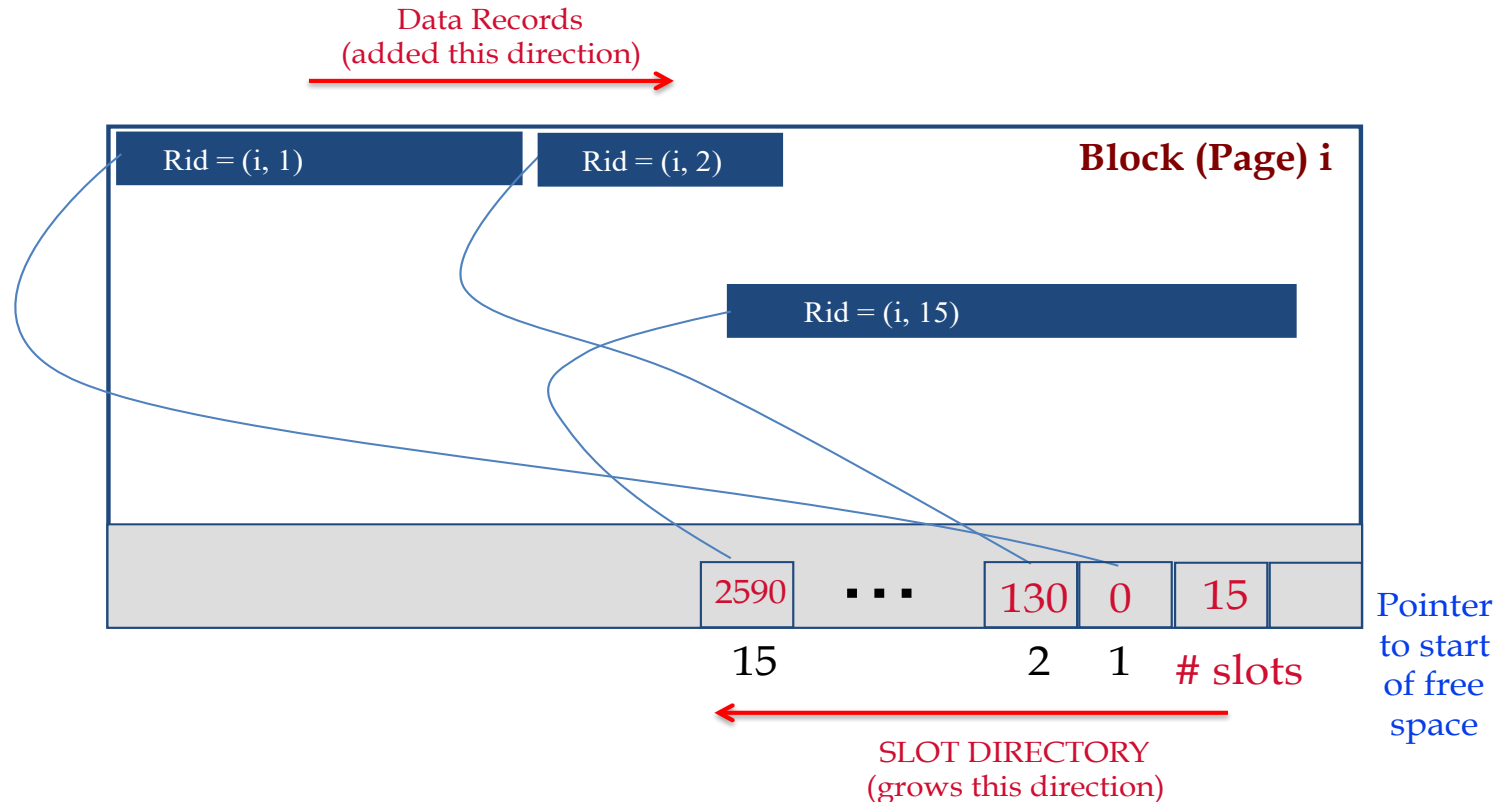
Question: How many records can fit in a disk Block

- We can do the math for fixed-length records

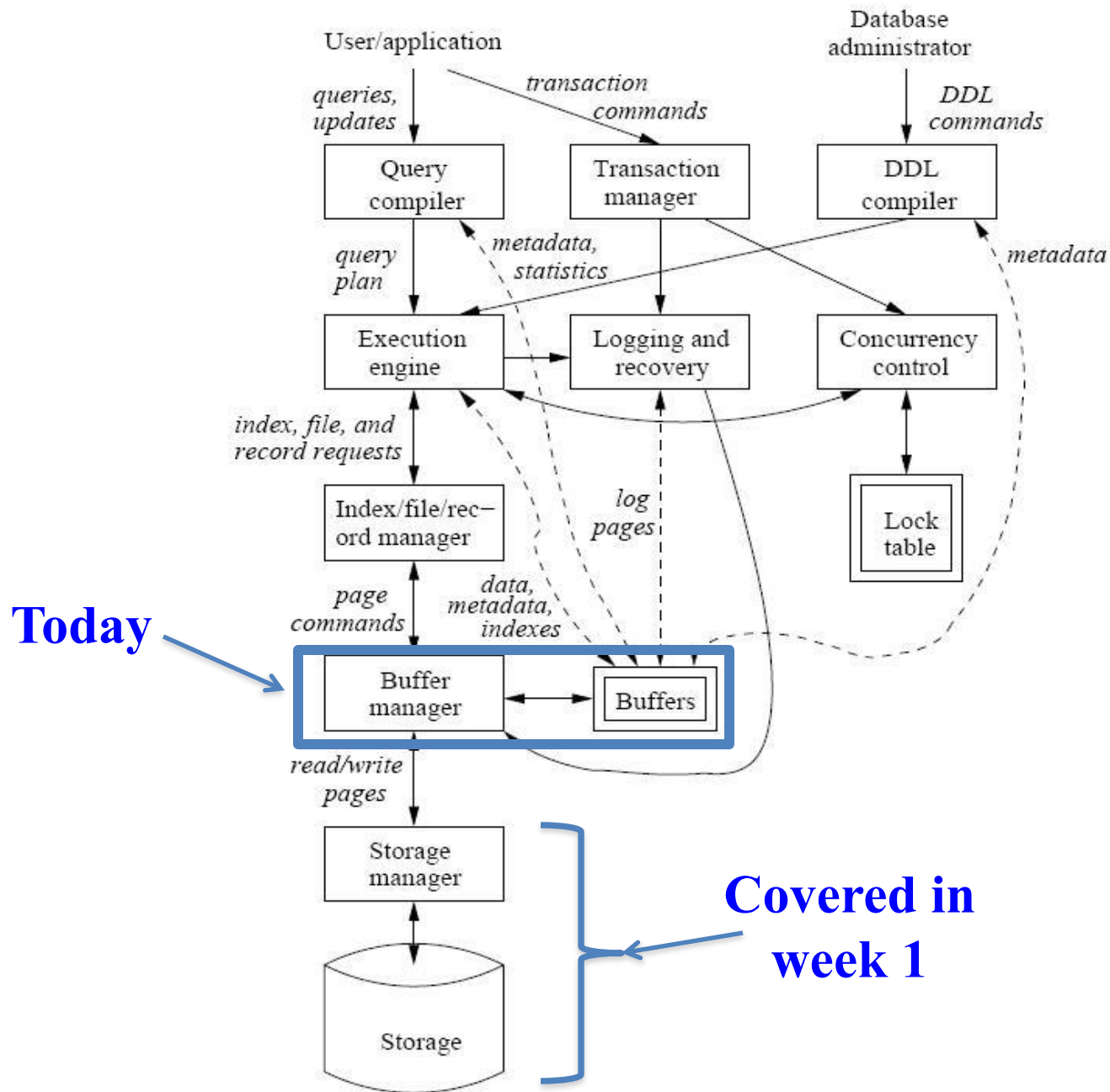


- For variable-length records, we can only estimate a range

Block Formats: Variable-Length Records

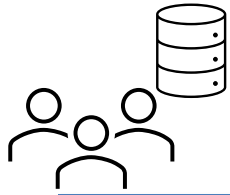


- The slot directory starts from one end and grows as needed
- The data records start from the other end (*No space is wasted*)



Learning objectives

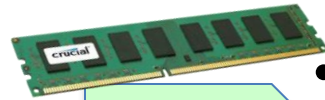
- Be familiar with terminology associated with buffer manager (e.g., pages, frames, replacement policy)
- Explain the flow of page requests to the buffer manager
- Explain metadata associated with frame and how it is updated



Buffer Manager

DB Higher-Level Components (E.g., Query Execution)

Buffer Manager



Main memory

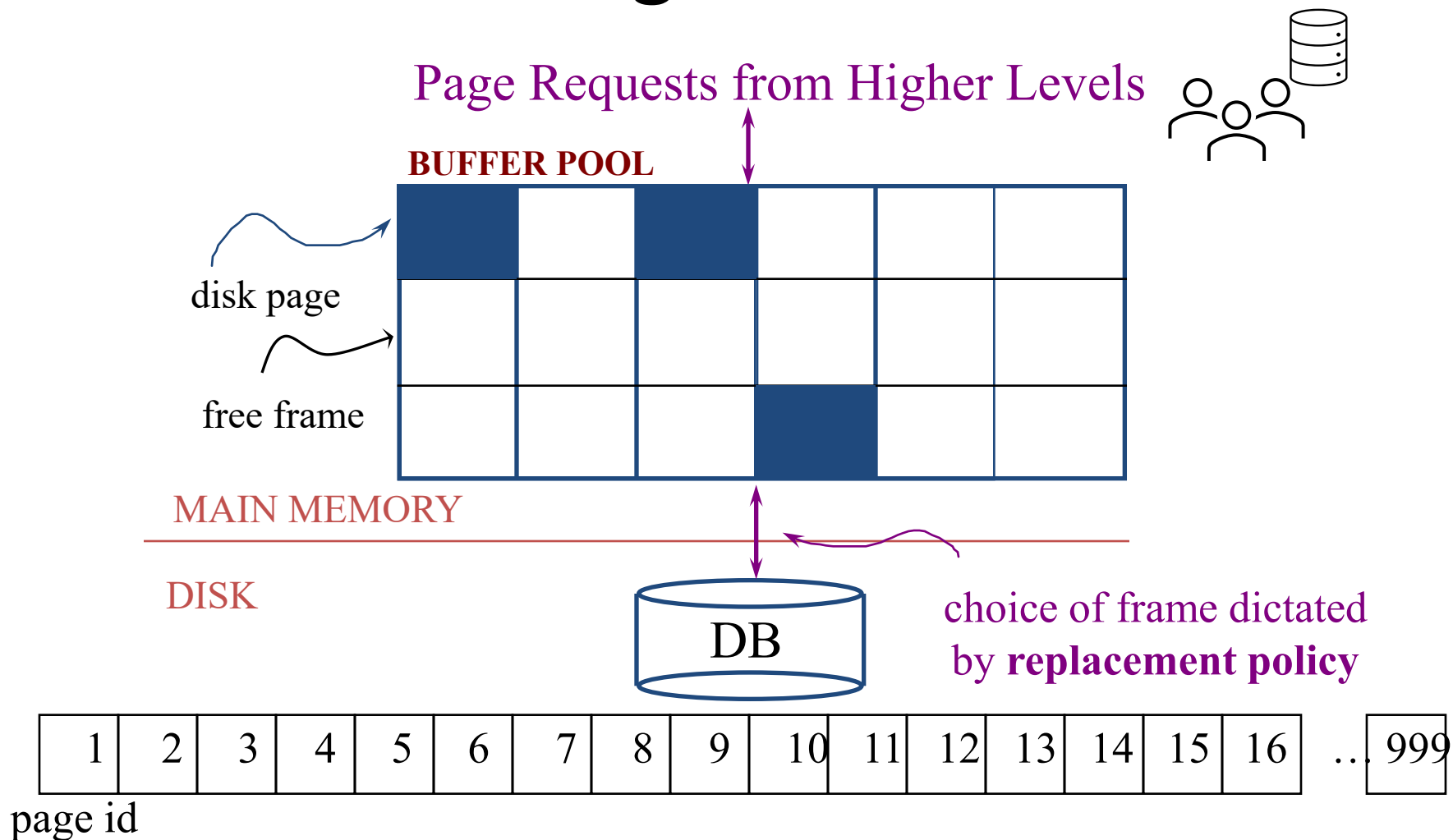
Storage Manager

Disk



- Higher-level components interact with Buffer Manager
- Buffer Manager manages **what blocks** should be in memory and for **how long**
- Any processing requires the data to be **in main memory**

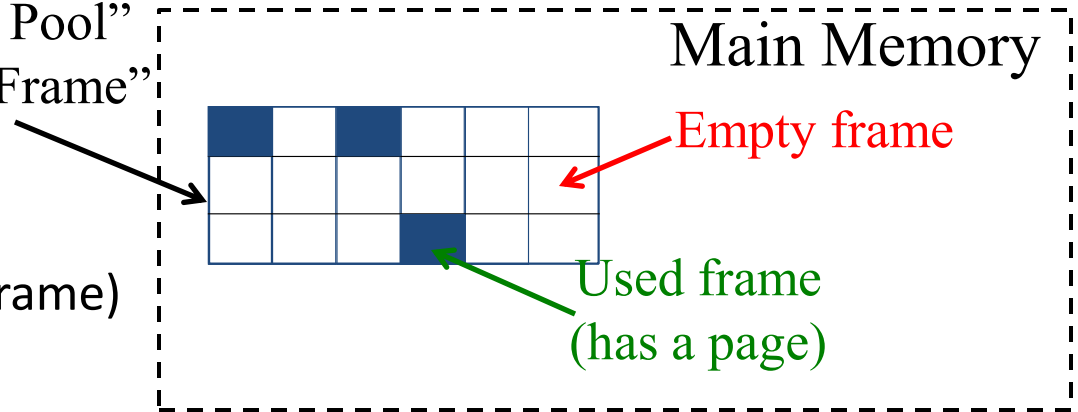
Buffer Management in a DBMS



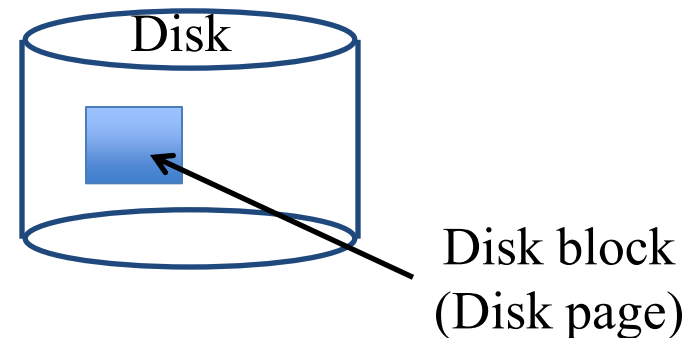
- **Buffer Pool** information table contains: $\langle \text{frame\#}, \text{disk-pageid}, \text{pin_count}, \text{dirty} \rangle$

Some Terminology

Array called “Buffer Pool”
Each entry is called “Frame”



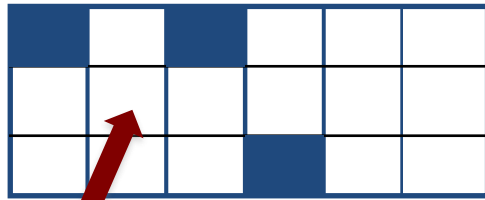
- Each entry in the Buffer Pool (Frame) can hold 1 disk block
- A disk block in memory is usually called “memory page”



- **Buffer Manager Keeps track of:**
 - Which frames are empty
 - Which disk page exists in which frame

• **Meta Data Information:** *<frame#, disk-pageid, pin_count, dirty>*

Buffer Pool



frame

Need record
 $\text{rid} = (\text{block } i, \text{slot } j)$

Extract the block Id from rid $\rightarrow i$

Is block i in
buffer pool?

No

Is there an
Empty frame?

Yes

(Frame k)

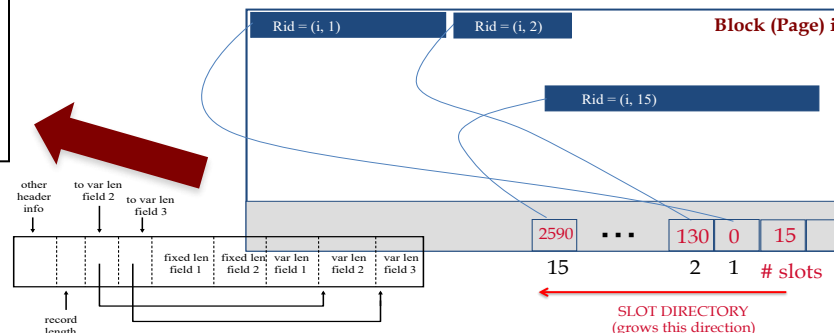
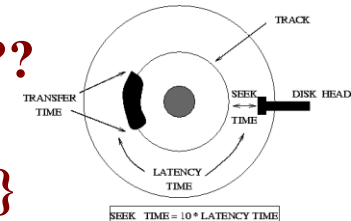
Get block i from disk

- Put block i in frame k
- Fill in the frame metadata

- Extract the slotId from rid $\rightarrow j$
- Read the record and return it to the caller

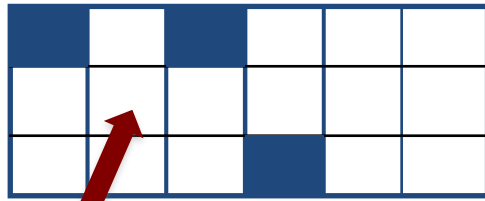
Do NOT get the block from disk unless we have a frame ready for it

How to get the block from disk??
File System maps block # $i \rightarrow$
{surface#, track#, start sector#}



At this level, it is a block-level check

Buffer Pool



frame

Need record
rid = (block i , slot j)

Extract the block Id from rid $\rightarrow i$

Is block i in
buffer pool?

No

Is there an
Empty frame?

Yes

(Frame k)

Get block i from disk

- Put block i in frame k
- Fill in the frame metadata

- Extract the slotId from rid $\rightarrow j$
- Read the record and return it to the caller

blockId $\leftarrow$ The id of the disk block stored in this frame

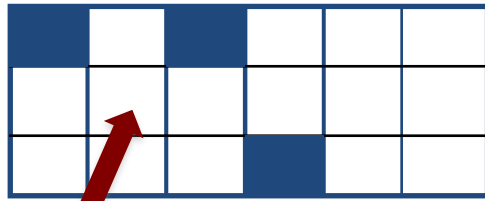
dirtyFlag $\leftarrow$ False (if the content has NOT changed)
 $\leftarrow$ True (if the content has changed)

pinnedFlag $\leftarrow$ False (it can be taken out if needed)
 $\leftarrow$ True (it has to stay in memory and cannot be taken out until un-pinned)

LTTimestamp $\leftarrow$ Last touched (read or write) timestamp

blockId $\leftarrow i$
 dirtyFlag $\leftarrow False$
 pinnedFlag $\leftarrow False$
 LTTimestamp $\leftarrow ?$

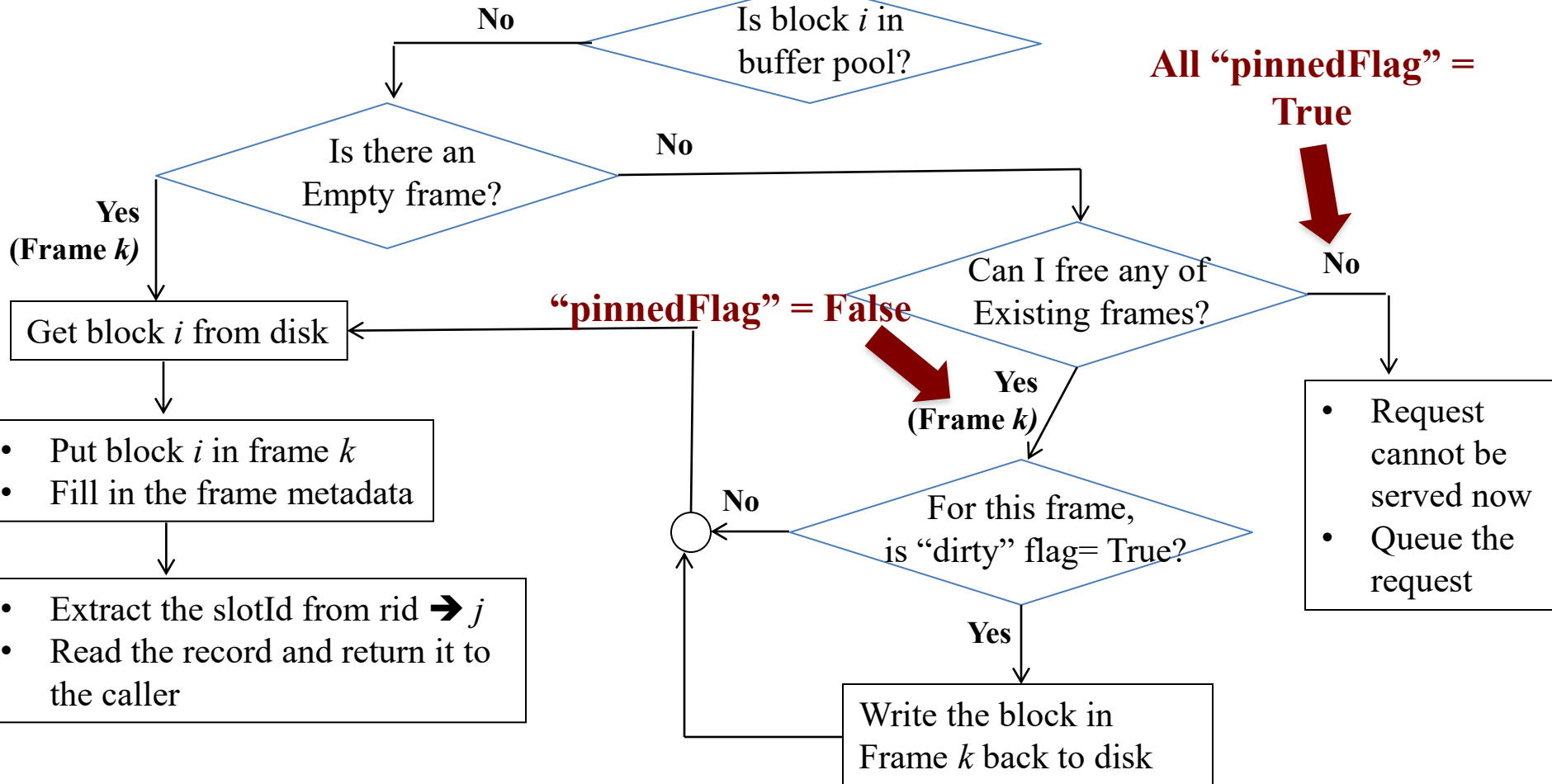
Buffer Pool

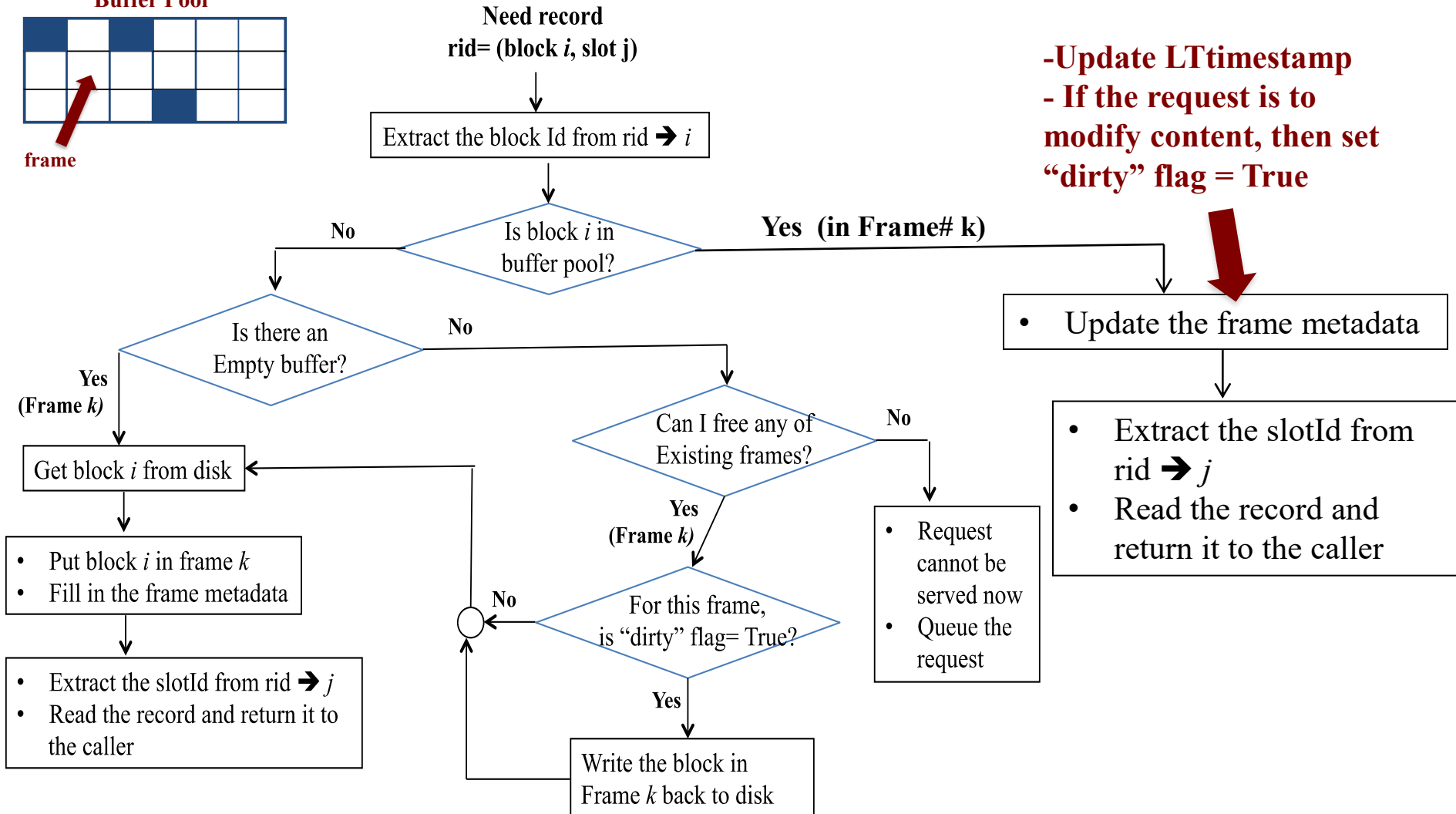
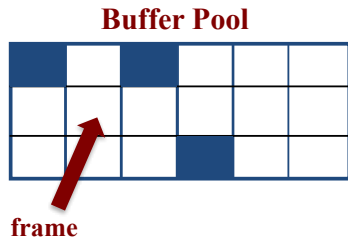


frame

Need record
 $rid = (\text{block } i, \text{slot } j)$

Extract the block Id from $rid \rightarrow i$





Question 1

Is block i in
buffer pool?

Naïve
Solution

Scan the array with each request → Check
“blockId” metadata → $O(n)$

Better Solution

Keep a hash table, given block Id (e.g., i) →
Returns the frame # (if exists) → $O(1)$

In this Sol. → No need to keep “blockId” inside
the frame metadata

Question 2

Is there an
Empty buffer?

Naïve
Solution

Scan the array with each request → look for “blockId = -1” → $O(n)$

Better Solution

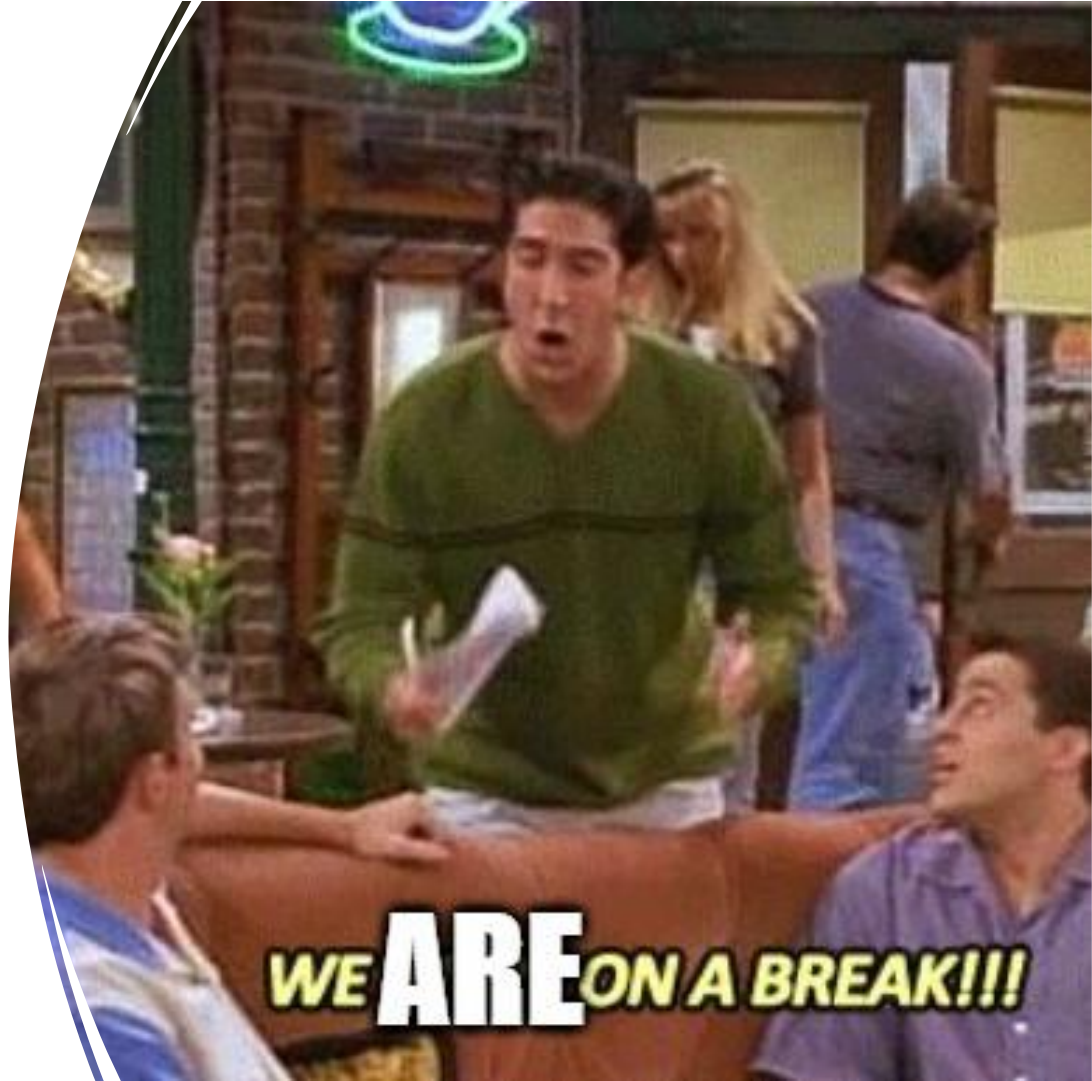
Keep a list of the empty frame# {1, 30, 50, ...}

Even Better
Solution

Keep a bitmap of the array size 111101001001...
0: Empty & 1: Used

Break

- Now we will take a few minutes break



CS4432: Database Systems II

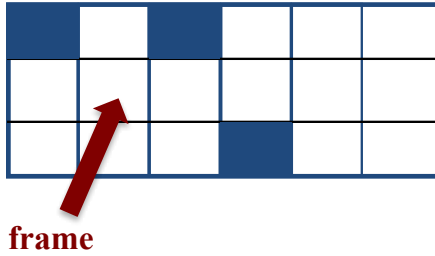
Buffer Manager

Part 2

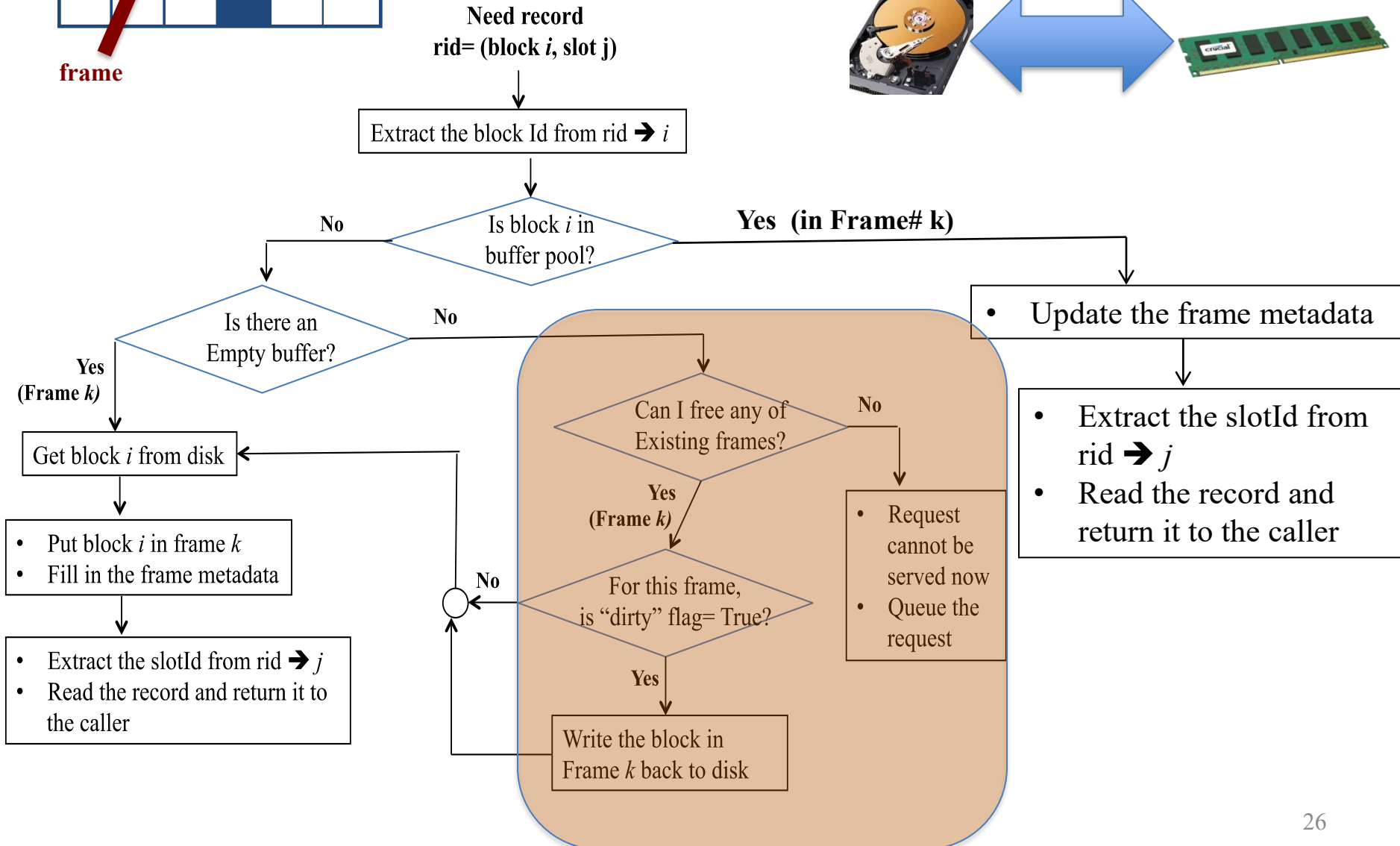
Learning objectives

- Be able to manually simulate LRU and clock replacement policies
- Understand the problem of sequential flooding
- Explain the importance of pinning pages and how the buffer manager deals with them

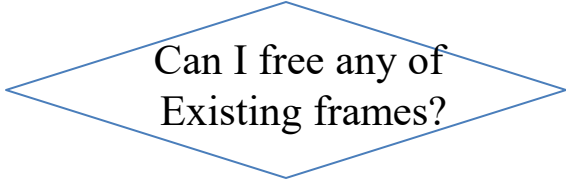
Buffer Pool



Summary



Question 3



Can I free any of
Existing frames?

- Choose a frame for *replacement*.
- Only “*un-pinned*” pages are candidates!

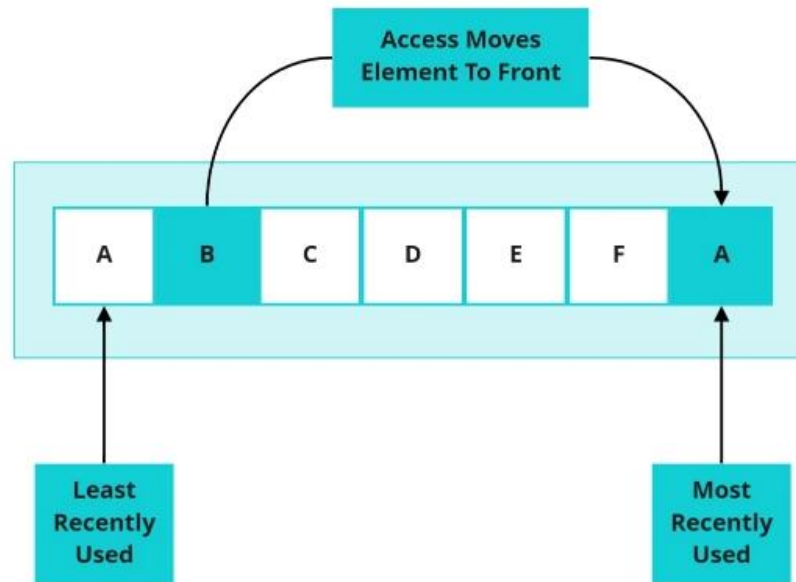
Buffer Replacement Policy

- Frame is chosen for replacement by a *replacement policy*:
 - Least-recently-used (LRU)
 - First-in-First-Out (FIFO),
 - Clock Policy
- Policy can have big impact on # of I/O's → depends on the *access pattern*.

May need additional metadata to be maintained by Buffer Manager

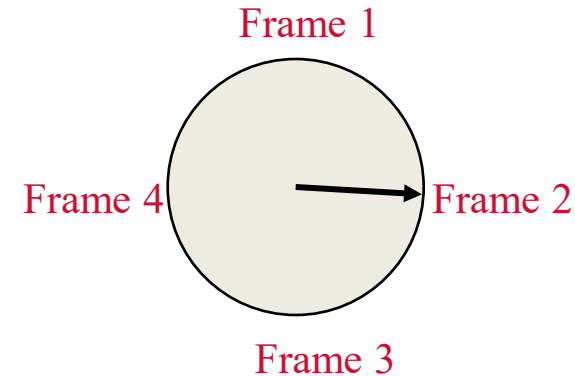
LRU Replacement Policy

- Least Recently Used (LRU)
 - for each frame in buffer pool, keep track of time when last *accessed* → (*LTtimestamp*)
 - replace the frame which has the oldest (earliest) time
 - very common policy: intuitive and simple
 - Works well for repeated accesses to popular pages



“Clock” Replacement Policy

- An approximation of LRU
- Each frame has
 - Pin count → If larger than 0, do not touch it
 - Ref bit (second chance bit) → 0 or 1
- Imagine frames organized into a cycle.
- A pointer rotates to find a candidate frame to free

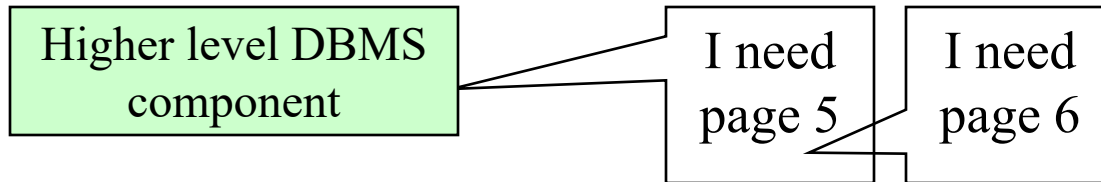


IF pin-count > 0 Then → Skip

IF (pin-count = 0) & (Ref = 1) → Set (Ref = 0) and skip (second chance)

IF (pin-count = 0) & (Ref = 0) → free and re-use

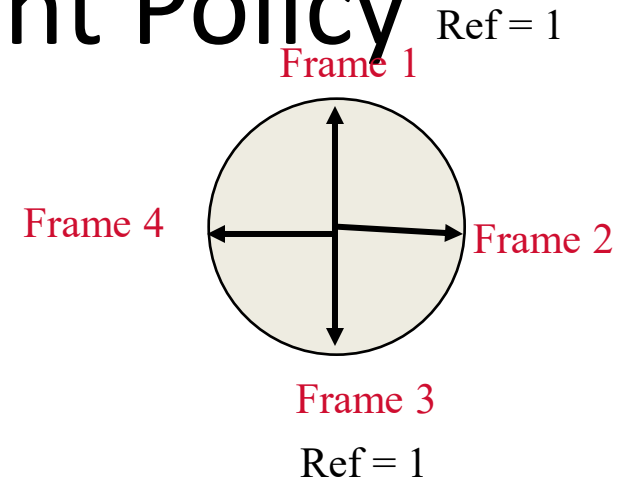
“Clock” Replacement Policy



Assume 5 pinned with insertion

Buf Mgr

```
do for each page in cycle {  
  if (pincount == 0 && ref bit is on)  
    turn off ref bit;  
  else if (pincount == 0 && ref bit is off)  
    choose this page for replacement;  
} until a page is chosen;
```



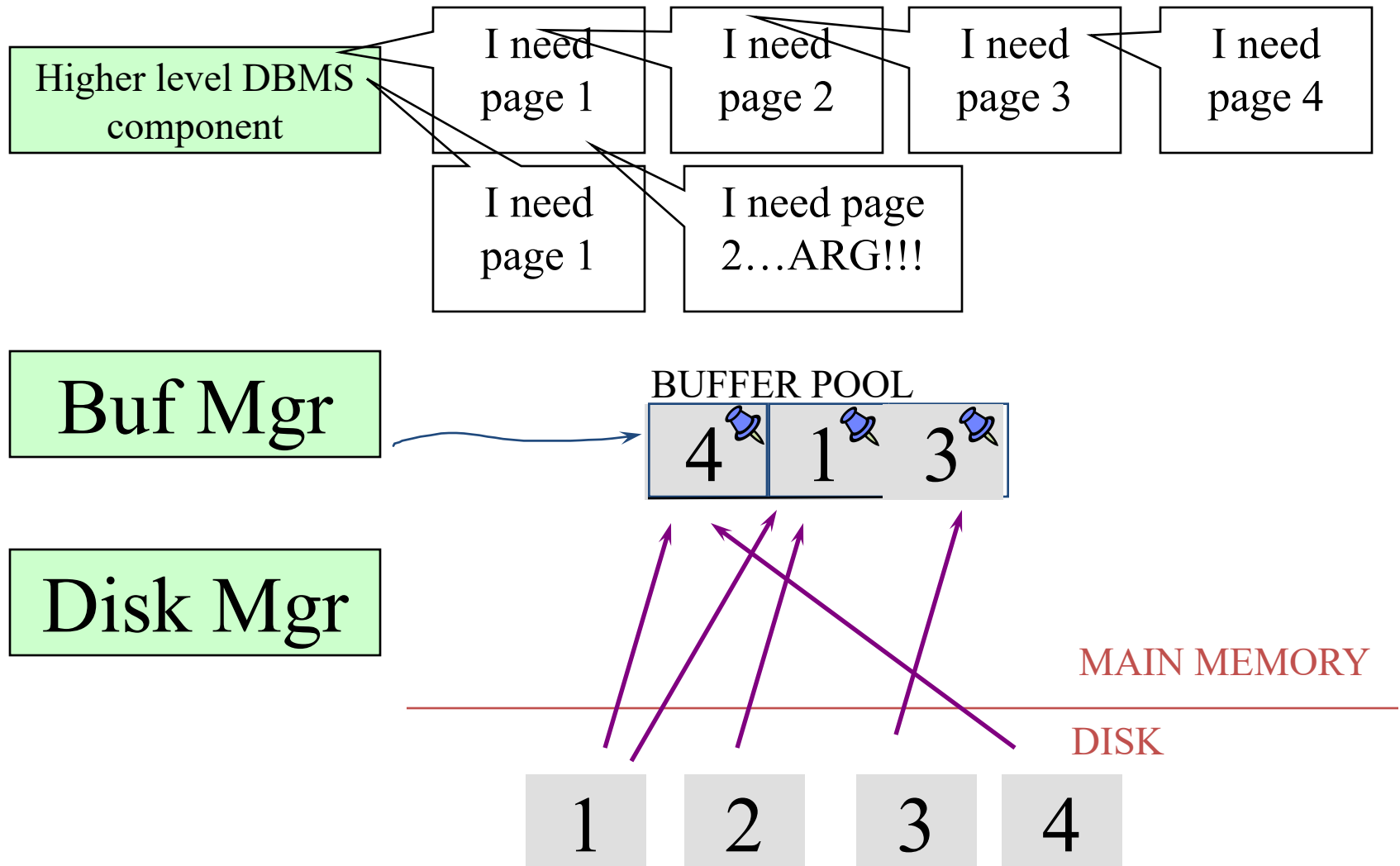
“Clock” Replacement Policy

- Need additional metadata:
 - second chance bit (Ref)
- Small overhead
- Every time a page is read, we set
Ref = 1
to indicate that it was recently used

Problem: Sequential Flooding

- Queries can involve repeated sequential scans
- # free buffer frames < # pages in file
means some page requests cause an I/O.
- High cost → Each access modifies the metadata
- Affects both LRU and Clock Replacement

Sequential flooding example with LRU



Other Replacement Policy

- Many other placement policies

Which policy
could mitigate
sequential
flooding?

Pin A Memory Page

- Can be a flag (T & F)
- Can be a counter (0 = unpinned)

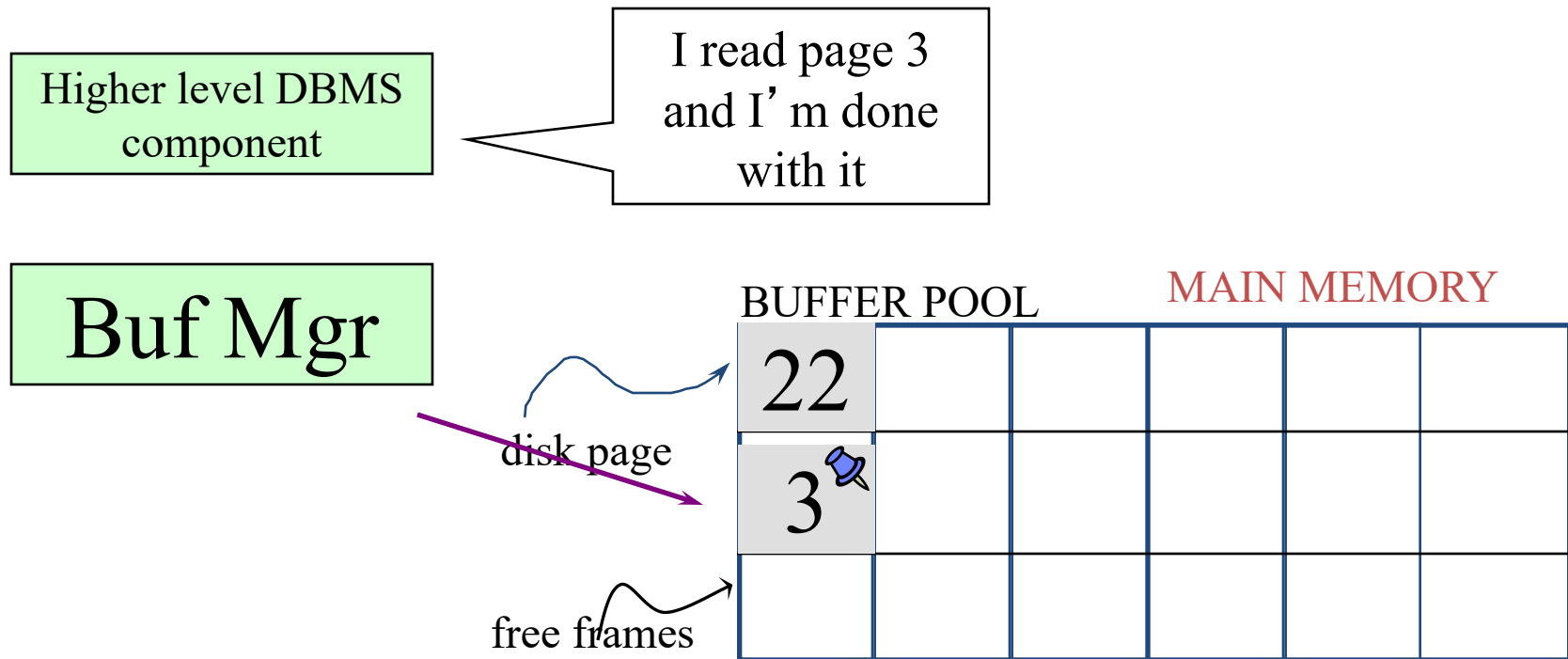
Pin this page

BUFFER POOL		MAIN MEMORY			
22					
3					

- Pinning a page means not to take from the memory until **un-pinned**
- **Why pin a page?**
 - Keep it until the transaction completes
 - Page is important (referenced a lot)
 - Recovery control
 - Concurrency control (they enforce certain order)

In practice, instead of pin flag (binary), pin count (int) is used

Releasing Unmodified Page



- Unpin the page (if you can)
- since page is not modified ➔ Just claim this frame# in *free list*
- No need to write back to disk

Releasing Modified page

Higher level DBMS component

I wrote on page 3 and I'm done with it

Buf Mgr

MAIN MEMORY

BUFFER POOL

22

3

disk page

free frames

3'

Disk Mgr

DISK

1

2

3'

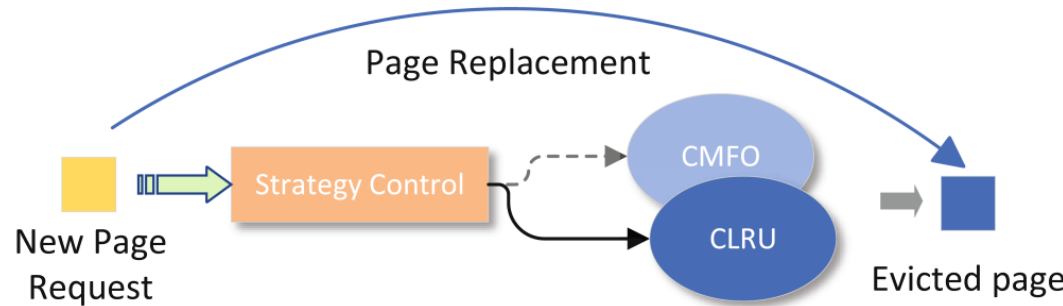
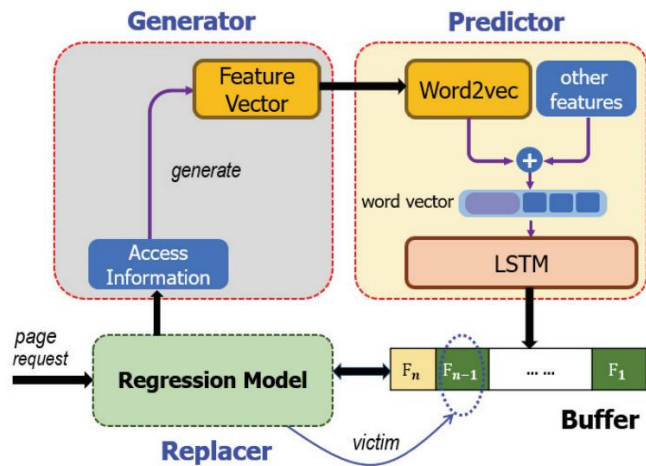
...

22

...

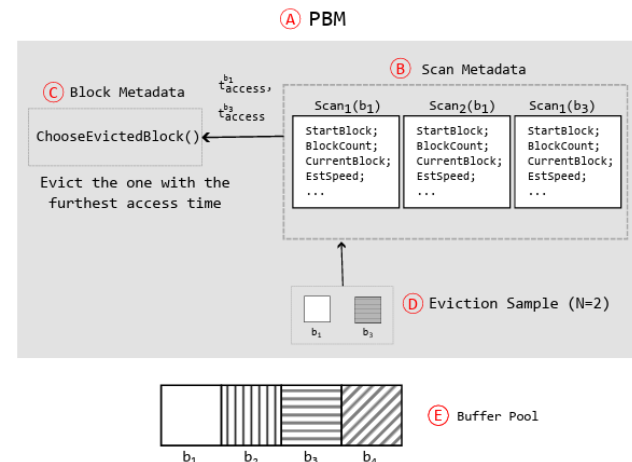
90

AI for Buffer Management?



RL-based buffer management [2]

Learned buffer management [1]



Predictive sampling-based buffer management [3]

[1] Yuan, Yigui, and Peiquan Jin. "Learned buffer management: A new frontier: Work-in-progress." *Proceedings of the 2021 International Conference on Hardware/Software Codesign and System Synthesis*. 2021. <https://dl.acm.org/doi/pdf/10.1145/3528114.3528118>

[2] Zhang, Sen, et al. "Reinforcement Learning-Enhanced Adaptive Buffer Replacement for Modern Databases." *Asia-Pacific Web (APWeb) and Web-Age Information Management (WAIM) Joint International Conference on Web and Big Data*. Singapore: Springer Nature Singapore, 2025.

[3] Khalaji, Mohammad, et al. "Sampling-Based Predictive Database Buffer Management." *Proceedings of the VLDB Endowment* 18.13 (2025): 5569-5581.